MALWARE ANALYSIS & DEFENSE

Laboratory guide — five hands-on exercises for beginning cybersecurity students, from safe file identification to a finished incident report.

COURSE LEVEL	COMPLETION TIME	PLATFORM	PREREQUISITES
Beginner	12–15 hours	Fedora · Ubuntu · Windows	Basic computer use and introductory command line

CONTENTS

1. Part 1 — Principles of malware analysis
2. Part 2 — Software and laboratory configuration
3. Lab 01 — Malware fundamentals and safe file identification
4. Lab 02 — Static analysis and file forensics
5. Lab 03 — Behavior simulation and process monitoring
6. Lab 04 — Detection with YARA and ClamAV
7. Lab 05 — Incident response and forensic reporting
8. Final assessment and grading rubric
9. Instructor guidance and troubleshooting
10. Study resources and course summary

HOW TO USE THIS GUIDE

Complete Part 2 once, then work through the labs in order — each lab builds on files created in the one before. Every exercise follows the same pattern: the **steps** you perform, the **commands** you type, the **expected result**, and **why** that result matters to an analyst.

Commands appear in shaded blocks. Type them exactly in the lab virtual machine's terminal — never on your everyday computer. A trailing `\` means the command continues on the next line. Command blocks are labelled **Linux · Terminal** or **Windows · PowerShell** — follow only the blocks for your platform.

PART 1

PRINCIPLES OF MALWARE ANALYSIS

What is malware?

Malware — short for *malicious software* — is software intentionally designed to compromise systems, steal information, disrupt operations, or perform other unauthorized activities.

Malware analysts investigate suspicious programs to understand what they do, how they operate, how they spread, and how defenders can detect and stop them.

Common malware categories

MALWARE TYPE	DESCRIPTION
Virus	Malicious code that attaches to another file or program and spreads when executed
Worm	Malware capable of propagating between systems on its own
Trojan	Malicious software disguised as legitimate software
Ransomware	Encrypts or otherwise denies access to data, commonly for extortion
Spyware	Secretly monitors or collects information
Rootkit	Conceals malicious activity or maintains privileged access
Keylogger	Software or hardware that records keystrokes
Botnet malware	Allows compromised devices to be remotely controlled
Cryptojacker	Uses computing resources for unauthorized cryptocurrency mining
Wiper	Designed to destroy or corrupt data

Table 1 — Malware categories referenced throughout the labs.

Four analysis methodologies

Analysts combine four complementary techniques. This beginner course emphasizes static analysis, safe behavioral observation, signature detection, and incident response.

01 · LABS 01–02

Static analysis

Examining a file without executing it: hashes, metadata, strings, embedded content and executable structure.

02 · LAB 03

Dynamic analysis

Observing a program while it runs in a controlled environment: processes, file activity and network behavior.

03 · LABS 03–04

Behavioral analysis

Identifying what a program does — creating files, launching processes, attempting connections.

04 · LAB 05

Memory & forensic analysis

Examining process memory, system artifacts and forensic evidence to reconstruct activity.

Principles every analyst follows

1. **Isolate first.** Analysis happens in a dedicated, disposable environment that cannot reach personal data or production networks.
2. **Evidence over assumptions.** A filename, an extension, or a single string is a clue, not a conclusion.
3. **Preserve before you analyze.** Hash and record evidence before handling it so you can later prove it has not changed.
4. **Least risky technique first.** Static analysis comes before execution; execution happens only in isolation.
5. **Document everything.** Commands, outputs, times and limitations go into your notes as you work — not afterwards.

THE ANALYST WORKFLOW PRACTISED IN THIS COURSE

IDENTIFY → PRESERVE → ANALYZE → OBSERVE → DETECT → INVESTIGATE → REPORT

PART 2

SOFTWARE & LABORATORY CONFIGURATION

Laboratory safety requirements

All exercises use harmless test files or explicitly identified simulation programs. Students must **not** download real malware from public repositories, execute unknown attachments, or analyze suspicious samples on their everyday computer.

- Use a dedicated virtual machine.
- Keep shared folders, shared clipboard and drag-and-drop disabled.
- Do not use personal credentials inside the laboratory.
- Take a clean VM snapshot before beginning.
- Use host-only networking or disable networking during simulations.
- Never present simulated indicators as evidence of an actual infection.

Note: a virtual machine reduces risk but does not provide an absolute security boundary.

Software used in this course

SOFTWARE	PURPOSE	USED IN	COST
VirtualBox (or another hypervisor)	Runs the isolated virtual machine	All labs	Free
Fedora Workstation or **Ubuntu Desktop**	Laboratory operating system	All labs	Free
Python 3	Runs the safe behavior and incident simulations	01, 03, 05	Free
ClamAV	Open-source antivirus scanning	01, 04	Free
YARA	Pattern matching for malware detection	04, 05	Free
file	Identifies a file's real format from its contents	01, 02	Free
strings (binutils)	Extracts readable text from files	02	Free
sha256sum (coreutils)	Calculates and verifies SHA-256 hashes	01, 02, 05	Free
xxd / **od**	Displays raw bytes in hexadecimal	02	Free
strace	Observes the system calls a process makes	03	Free
tcpdump	Captures network traffic from the command line	Capstone (optional)	Free
Wireshark	Graphical packet analysis	Capstone (optional)	Free
jq	Reads and filters JSON event data	05	Free
git	Retrieves course materials and tracks your notes	Setup	Free

Table 2 — Laboratory software. Everything is free and open source except the hypervisor, which has free options.

Windows equivalents

Windows has no built-in `file`, `strings` or `strace`. The Windows track replaces each Linux tool as follows. Everything is free.

LINUX TOOL	WINDOWS REPLACEMENT	SOURCE
`file`	**file.exe, bundled with Git for Windows**	git-scm.com
`sha256sum`	**Get-FileHash**	Built into PowerShell
`stat`	**Get-Item, Get-Acl**	Built into PowerShell
`strings`	**strings64.exe**	Sysinternals Suite
`grep`	**Select-String**	Built into PowerShell
`xxd / od`	**Format-Hex**	Built into PowerShell
`ps / top`	**Get-Process, Get-CimInstance, Task Manager**	Built in
`watch`	**A PowerShell while-loop**	Built in
`strace`	**Process Monitor (Procmon64.exe)**	Sysinternals Suite
`nano`	**Notepad**	Built in
`clamscan / freshclam`	**ClamAV for Windows**	clamav.net
`yara`	**yara64.exe**	github.com/VirusTotal/yara
`jq`	**ConvertFrom-Json, or jq.exe**	PowerShell / winget
`tcpdump`	**pktmon, or Wireshark with Npcap**	Built in / wireshark.org
—	**Microsoft Defender Antivirus**	Built into Windows

Table 3 — Linux tools and their Windows equivalents.

Step 1 · Install the hypervisor

The hypervisor runs a complete, separate computer (the *guest*) inside your real computer (the *host*).

1. Open https://www.virtualbox.org/ and choose **Downloads**.
2. Download the package for your host operating system (Windows, macOS or Linux).
3. Run the installer and accept the default options. On Windows, approve the network-driver prompts; on macOS, allow the Oracle system extension in *System Settings → Privacy & Security* if asked.
4. Launch VirtualBox to confirm it opens.

EXPECTED RESULT

The VirtualBox Manager window opens with an empty machine list.

WHY

Everything that follows happens inside a guest. If the hypervisor does not start, enable hardware virtualization (Intel VT-x / AMD-V) in your computer's firmware settings before continuing.
Windows guests need more resources. Plan for a host with at least 16 GB RAM and 80 GB of free disk space.

Step 2 · Download the laboratory operating system

- **Fedora Workstation:** https://fedoraproject.org/workstation/ → download the x86_64 Live ISO (or aarch64 on Apple Silicon Macs).
- **Ubuntu Desktop (LTS):** https://ubuntu.com/download/desktop → download the current LTS release ISO.
- **Windows 11 Enterprise (evaluation):** https://www.microsoft.com/en-us/evalcenter/evaluate-windows-11-enterprise → choose the 64-bit ISO, complete the short registration form and download. The evaluation is free for 90 days — long enough for the course, and restorable from your snapshot.

Save the ISO somewhere you can find it, such as your Downloads folder. You only need one distribution.

Step 3 · Create and harden the virtual machine

1. In VirtualBox, click **New**. Name the machine `MalwareLab` and select the ISO you downloaded. Choose *Skip unattended installation* if offered.
2. Hardware: **4096 MB** RAM minimum and **2** virtual CPUs.
3. Hard disk: create a virtual disk of **40 GB**.
4. Click **Finish**, then open **Settings** for the new machine.
5. **General → Advanced:** set *Shared Clipboard* and *Drag'n'Drop* to **Disabled**.
6. **Shared Folders:** confirm the list is empty.
7. **Network → Adapter 1:** leave as **NAT** for now — you need internet access to install tools. You will isolate the VM in Step 9.
8. Click **OK**, then **Start** the machine and follow the installer (default disk layout; create a non-administrator-named user such as `student` with administrator rights). Do not sign in to any personal accounts.
9. After installation, reboot into the new system and remove the ISO if VirtualBox does not eject it automatically.

WINDOWS 11 GUEST — DIFFERENT SETTINGS

1. In **New**, set *Type* to **Microsoft Windows** and *Version* to **Windows 11 (64-bit)**. VirtualBox 7 then enables EFI, Secure Boot and a virtual TPM 2.0 — Windows 11 setup refuses to install without them.

2. Give the VM **8192 MB** RAM, **2** virtual CPUs (4 if your host has 8 or more cores) and a **64 GB** disk.
3. Apply the same hardening as above: Shared Clipboard and Drag'n'Drop **Disabled**, no Shared Folders, Adapter 1 on **NAT**.
4. Start the VM and run setup: choose **Windows 11 Enterprise Evaluation** → **Custom: Install Windows only** → the empty disk.
5. At the account screen choose **Sign-in options** → **Domain join instead**. This creates a local account — name it `student`. Never sign in with a personal Microsoft account inside the lab.
6. Decline the optional privacy prompts (location, diagnostic data, advertising ID).
7. On the desktop, open **Settings** → **Windows Update** and install updates until none remain, rebooting as required.
8. Open **File Explorer** → **View** → **Show** and tick **File name extensions**. Windows hides extensions by default — exactly what lets `invoice.pdf.exe` pass as a PDF.

EXPECTED RESULT

A Linux desktop boots inside the VirtualBox window. Copy-and-paste between host and guest does not work.

WHY

Shared clipboards and folders are bridges between guest and host. Closing them means anything that happens inside the lab stays inside the lab.

Step 4 · Update the system and install the laboratory tools

Open the **Terminal** application inside the VM and run the commands for your distribution. Enter your password when `sudo` asks for it.

FEDORA

```
sudo dnf update -y
sudo dnf install -y \
  python3 python3-pip \
  clamav clamav-update \
  yara file binutils vim-common \
  strace tcpdump wireshark jq git
```

UBUNTU

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
  python3 python3-pip \
  clamav clamav-freshclam \
  yara file binutils xxd \
  strace tcpdump wireshark jq git
```

If the Wireshark installer asks whether non-superusers should be able to capture packets, choose **Yes**.

WINDOWS

Windows installs some tools with **winget** (its built-in package manager) and the rest from their official download pages. Right-click **Start → Terminal (Admin)** to open an administrator PowerShell window.

4a · Python, Git, Wireshark and jq

```
winget install --id Python.Python.3.12 -e --accept-package-agreements --accept-
source-agreements
winget install --id Git.Git -e
winget install --id WiresharkFoundation.Wireshark -e
winget install --id jqlang.jq -e
```

When the Wireshark installer opens, keep **Install Npcap** ticked — Wireshark cannot capture traffic on Windows without it. Close and reopen the terminal afterwards so the new PATH entries load.

4b · ClamAV

1. Open https://www.clamav.net/downloads and find the **Windows** section.
2. Download the latest `clamav-1.x.x.win.x64.msi`.
3. Double-click it and accept the defaults. ClamAV installs to `C:\Program Files\ClamAV`. You configure it in Step 6.

4c · YARA

Open https://github.com/VirusTotal/yara/releases, expand **Assets** under the latest release and download `yara-v4.x.x-xxxx-win64.zip`. Then extract it:

```
New-Item -ItemType Directory -Force -Path C:\Tools\yara
Expand-Archive -Path $HOME\Downloads\yara-*-win64.zip -DestinationPath C:\Tools\yara
-Force
Get-ChildItem C:\Tools\yara
```

The folder should contain `yara64.exe` and `yarac64.exe`.

4d · Sysinternals Suite (strings and Process Monitor)

Download the Sysinternals Suite zip from https://learn.microsoft.com/sysinternals/downloads/sysinternals-suite, then extract it:

```
New-Item -ItemType Directory -Force -Path C:\Tools\Sysinternals
Expand-Archive -Path $HOME\Downloads\SysinternalsSuite.zip -DestinationPath
C:\Tools\Sysinternals -Force
```

4e · Make the tools available in every PowerShell window

Close the admin window and open a **normal** PowerShell window (Start → Terminal). Allow your own scripts to run, then create a PowerShell profile:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
New-Item -ItemType File -Force -Path $PROFILE
notepad $PROFILE
```

Type these lines into Notepad, save with Ctrl+S and close it:

```
$env:Path += ";C:\Program Files\ClamAV;C:\Tools\yara;C:\Tools\Sysinternals"
Set-Alias -Name file -Value "C:\Program Files\Git\usr\bin\file.exe"
Set-Alias -Name yara -Value "C:\Tools\yara\yara64.exe"
Set-Alias -Name strings -Value "C:\Tools\Sysinternals\strings64.exe"
```

WHY

The profile runs every time PowerShell opens. It puts the tool folders on the PATH and creates the short names `file`, `yara` and `strings`, so Windows commands in this guide read almost like their Linux equivalents. `RemoteSigned` lets scripts you create locally run, while still blocking unsigned scripts downloaded from the internet.

If a package name differs on your release, look it up with `dnf search <name>` or `apt search <name>` before proceeding.

Step 5 · Verify the installation

LINUX · TERMINAL

```
python3 --version
clamscan --version
yara --version
strace --version
jq --version
xxd -v
```

WINDOWS · POWERSHELL

```
python --version
git --version
clamscan --version
yara --version
file --version
strings -accepteula -nobanner $PROFILE
jq --version
```

Windows result: Each command prints a version (the `strings` line prints the text of your profile; `-accepteula` accepts the Sysinternals licence once so it never prompts again). If a command is *not recognized*, re-check Step 4e and reopen PowerShell. If `python` opens the Microsoft Store, see Troubleshooting.

EXPECTED RESULT

Each command prints a version number. None reports *command not found*.

WHY

Once the VM is isolated you can no longer download missing tools. Confirming every tool now prevents a dead end mid-lab.

Step 6 · Configure ClamAV and download signatures

ClamAV only detects what its signature database knows about. Download the database while internet access is still available.

FEDORA

```
sudo freshclam
```

If FreshClam reports a configuration error mentioning `Example`, open `/etc/freshclam.conf` with `sudo nano /etc/freshclam.conf`, place a `#` in front of the line that reads `Example`, save, and run the command again.

UBUNTU

Ubuntu runs an automatic updater service. Stop it, update manually, then restart it, so two updaters do not compete:

```
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
```

WINDOWS

ClamAV for Windows ships with sample configuration files only. Create working copies, then download signatures. In an **administrator** PowerShell window:

```
cd "C:\Program Files\ClamAV"
Copy-Item .\conf_examples\freshclam.conf.sample .\freshclam.conf
Copy-Item .\conf_examples\clamd.conf.sample .\clamd.conf
notepad .\freshclam.conf
```

Near the top of the file, find the line that reads `Example` and type `#` in front of it. Save and close. Repeat for `notepad .\clamd.conf`. The *Example* line exists to stop the program running on an unedited sample. Then download the signatures:

```
.\freshclam.exe
```

Windows result: FreshClam creates `C:\Program Files\ClamAV\database` and downloads the same three database files as on Linux.

Confirm the database loaded — the version line now includes a database number and date:

```
clamscan --version
```

EXPECTED RESULT

FreshClam reports that `main.cvd`, `daily.cvd` and `bytecode.cvd` are downloaded or up to date. The version output reads like `ClamAV 1.x.x/27xxx/<date>`.

WHY

Without signatures the scanner has nothing to compare files against, and the EICAR exercises in Labs 01 and 04 would produce a misleading "clean" result.

Step 7 · Set permissions & protections

LINUX

Add your account to the `wireshark` group so you can use Wireshark without running it as root, then log out and back in:

```
sudo usermod -aG wireshark $USER
groups
```

After logging back in, `groups` should list `wireshark`. Running a capture tool as an ordinary user limits the damage a parsing bug could cause.

WINDOWS

Packet-capture permission on Windows comes from Npcap, installed with Wireshark in Step 4a. The setting that matters here is **Microsoft Defender Antivirus**, which is built in and always on. Left alone, Defender deletes the EICAR test file the moment Lab 01 creates it, and you cannot practise with ClamAV. Tell Defender to skip the lab workspace — and nothing else.

In an **administrator** PowerShell window (signed in as `student`, so `$HOME` is your own folder):

```
Add-MpPreference -ExclusionPath "$HOME\MalwareLab"
Get-MpPreference | Select-Object -ExpandProperty ExclusionPath
```

EXPECTED RESULT · WINDOWS

The second command prints `C:\Users\student\MalwareLab`.

WHY

An exclusion is a deliberate blind spot. It is acceptable only inside this isolated, disposable VM — **never add exclusions on your host or any real computer**. Attackers add Defender exclusions themselves to hide malware (MITRE ATT&CK T1562.001, *Impair Defenses*), which is why investigators check this list. You will see the consequences in Labs 04 and 05.

Step 8 · Create the laboratory workspace

LINUX · TERMINAL

```
mkdir -p ~/MalwareLab/{samples,logs,reports,rules,scripts,evidence}
cd ~/MalwareLab
find . -maxdepth 1 -type d
```

WINDOWS · POWERSHELL

```
"samples","logs","reports","rules","scripts","evidence" | ForEach-Object { New-Item -
ItemType Directory -Force -Path "$HOME\MalwareLab\$_" }
cd $HOME\MalwareLab
Get-ChildItem -Directory
```

Windows result: Six folders listed. Throughout the labs, Linux `~/MalwareLab` means `$HOME\MalwareLab` (`C:\Users\student\MalwareLab`) on Windows. The Python scripts use `Path.home()`, so they work unchanged on both.

FOLDER	HOLDS
`samples/`	Training files you analyze
`logs/`	Tool output such as strace logs
`reports/`	Your written lab reports
`rules/`	YARA detection rules
`scripts/`	The safe Python simulators
`evidence/`	Simulated incident evidence

The `find` command should list all six folders. Keeping samples, evidence and reports apart mirrors how real analysts separate untrusted material from their own work.

Step 9 · Snapshot and isolate the VM

1. Shut down the VM: `sudo poweroff` on Linux, or **Start → Power → Shut down** on Windows.
2. In VirtualBox, select the machine → **Snapshots** → **Take**. Name it `Malware-Lab-Clean`.
3. Open **Settings → Network → Adapter 1** and change *Attached to* from NAT to **Host-only Adapter** (or untick *Enable Network Adapter*).
4. Start the VM and confirm the internet is unreachable:

LINUX · TERMINAL

```
ping -c 2 9.9.9.9
```

WINDOWS · POWERSHELL

```
Test-NetConnection 9.9.9.9
```

Windows result: A warning that the ping failed and `PingSucceeded : False`. Take the snapshot after Step 7, so the Defender exclusion is part of the clean state.

EXPECTED RESULT

The ping fails with *Network is unreachable* or 100% packet loss, and the snapshot appears in the Snapshots list.

WHY

The snapshot is your undo button: if anything goes wrong, **Restore** returns the VM to this exact clean state in seconds. Isolation ensures no simulated activity can reach a real network. To update signatures later, temporarily switch back to NAT, update, then isolate again.

LAB 01

MALWARE FUNDAMENTALS & SAFE FILE IDENTIFICATION

Duration · 2–3 hours | Difficulty · Beginner | Linux · file, sha256sum, python3, clamscan | Windows · Get-FileHash, Format-Hex, Defender

Learning objectives

1. Explain the major categories of malware.
2. Recognize suspicious file characteristics.
3. Identify file types using Linux utilities.
4. Generate and compare cryptographic file hashes.
5. Use the EICAR antivirus test file.
6. Explain why detection alerts require investigation.

WINDOWS TRACK — READ BEFORE EVERY LAB

- Run lab commands in a **normal** PowerShell window (Start → Terminal). Use an administrator window only where a step says so.
- Each **Windows · PowerShell** block replaces the **Linux · Terminal** block directly above it.
- `~/MalwareLab` = `$HOME\MalwareLab`; `python3` = `python`; nano = Notepad (`notepad <file>`, answer **Yes** to create it).
- Windows ends text lines with two bytes (CR LF) where Linux uses one (LF), so some sizes and hashes differ from Linux classmates — Exercise 2 explains why.
- Windows-only exercises (3b, 5b, 4b) are marked and use Microsoft Defender, Windows' built-in antivirus.

Exercise 1 · File types are not filenames

A file extension is a label anyone can change. A file named `invoice.pdf` might not contain PDF data at all. Analysts use identification tools that inspect the contents.

1. Move to the samples folder and create a harmless text file:

LINUX · TERMINAL

```
cd ~/MalwareLab/samples
echo "Cybersecurity malware training sample" > training.txt
```

WINDOWS · POWERSHELL

```
cd $HOME\MalwareLab\samples
Set-Content -Path training.txt -Value "Cybersecurity malware training sample" -
Encoding ascii
```

2. Copy it under a misleading name:

LINUX · TERMINAL

```
cp training.txt suspicious.exe
```

WINDOWS · POWERSHELL

```
Copy-Item training.txt suspicious.exe
```

3. Inspect both files:

LINUX · TERMINAL

```
file training.txt
file suspicious.exe
```

WINDOWS · POWERSHELL

```
file training.txt
file suspicious.exe
Format-Hex suspicious.exe | Select-Object -First 1
```

Windows result: Both files report `ASCII text, with CRLF line terminators`. `Format-Hex` shows the first bytes `43 79 62 65` ("Cybe") — not `4D 5A` ("MZ"), the header every genuine Windows program starts with. Do **not** double-click `suspicious.exe`: Windows decides how to open a file by its extension, so it would try to run it as a program — the very behavior attackers exploit with renamed files.

EXPECTED RESULT

Both lines end in `ASCII text` — despite one being named `.exe`.

WHY

`file` reads the first bytes of the file ("magic numbers") instead of trusting the name. Attackers routinely rename files — for example `invoice.pdf.exe` — so analysts always check content.

STUDENT QUESTIONS

1. Why is a filename extension unreliable?
2. Does an `.exe` extension automatically mean a file is malicious?
3. What additional evidence would help determine whether a file is suspicious?

Exercise 2 · Generate cryptographic hashes

A cryptographic hash is a fixed-length value calculated from a file's contents. Analysts use SHA-256 hashes to identify and compare files.

1. Hash the file and write the 64-character value in your notes:

LINUX · TERMINAL

```
sha256sum training.txt
```

WINDOWS · POWERSHELL

```
Get-FileHash -Path training.txt -Algorithm SHA256 | Format-List
```

2. Append one line to the file, then hash it again:

LINUX · TERMINAL

```
echo "Additional training content" >> training.txt
sha256sum training.txt
```

WINDOWS · POWERSHELL

```
Add-Content -Path training.txt -Value "Additional training content" -Encoding ascii
Get-FileHash -Path training.txt -Algorithm SHA256 | Format-List
(Get-Item training.txt).Length
```

Windows result: The two hashes differ completely. Your hash will *also* differ from a Linux classmate's for the same sentences: Windows wrote each line ending as two bytes (`0D 0A`) where Linux writes one (`0A`) — compare the `Length` values. Different bytes, different hash. `Get-FileHash` prints uppercase hexadecimal and `sha256sum` lowercase; case does not change the value.

3. Compare the two values side by side.

EXPECTED RESULT

The two hashes are completely different, not just slightly different.

WHY

Any change — even one byte — produces an unrelated hash. That makes a hash a reliable fingerprint for an *exact* file, and explains why attackers can evade hash-based blocklists simply by modifying a sample.

STUDENT QUESTIONS

1. Why did the hash change?
2. How can hashes help identify malware samples?
3. Does a matching hash prove that a file is malicious?

Exercise 3 · Create the EICAR antivirus test file

The EICAR test file is an industry-standard, harmless string that every antivirus product agrees to detect. **It is not malware.** Python builds it from two parts so it is not mistyped.

1. From `~/MalwareLab/samples`, run:

LINUX · TERMINAL

```
python3 - <<'PY'
from pathlib import Path

parts = [
    "X5O!P%@AP[4\\PZX54(P^)7CC)7}",
    "$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*"
]

Path("eicar.com").write_bytes(
    "".join(parts).encode("ascii")
)
PY
```

WINDOWS · POWERSHELL

PowerShell has no heredoc, so pipe a here-string into Python instead. Confirm the Defender exclusion from Part 2, Step 7 is in place first.

```
@'
from pathlib import Path

parts = [
    "X5O!P%@AP[4\\PZX54(P^)7CC)7}",
    "$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*"
]

Path("eicar.com").write_bytes(
    "".join(parts).encode("ascii")
)
'@ | python -
```

2. Confirm the size, then scan it:

LINUX · TERMINAL

```
wc -c eicar.com
clamscan eicar.com
```

WINDOWS · POWERSHELL

```
(Get-Item eicar.com).Length
clamscan eicar.com
```

Windows result: 68, then the same ClamAV `FOUND` result as on Linux. If Python raises `PermissionError` or `eicar.com` vanishes, Defender removed it because the exclusion is missing — still a successful detection, just by a different engine. Add the exclusion and try again.

EXPECTED RESULT

`wc` reports **68** bytes. ClamAV prints a line such as `eicar.com: Win.Test.EICAR_HDB-1 FOUND` and the summary shows `Infected files: 1`.

WHY

A detection proves the scanner and its signature database are working — without any real risk. If nothing is detected, investigate the database (Part 2, Step 6) rather than assuming the antivirus is ineffective or the file is dangerous.

Exercise 3b · Watch Microsoft Defender respond

Windows only

Windows has its own antivirus with real-time protection. Copy the test file *outside* the excluded folder and watch what happens.

WINDOWS · POWERSHELL

```
Copy-Item eicar.com $HOME\Documents\eicar-defender.com
Start-Sleep -Seconds 5
Test-Path $HOME\Documents\eicar-defender.com
```

Then, in an **administrator** PowerShell window, read Defender's record of the event:

```
Get-MpThreat | Select-Object ThreatName, SeverityID, IsActive
Get-MpThreatDetection | Sort-Object InitialDetectionTime -Descending | Select-Object
-First 1 | Format-List InitialDetectionTime, Resources, ActionSuccess
```

Finally, open **Windows Security → Virus & threat protection → Protection history** and find the same event. Do **not** choose *Allow on device* or *Restore*.

EXPECTED RESULT · WINDOWS

`Copy-Item` either fails with *"Operation did not complete successfully because the file contains a virus or potentially unwanted software"*, or the copy disappears within seconds. `Test-Path` returns `False`, and `Get-MpThreat` lists `Virus:DOS/EICAR_Test_File`.

WHY

Real-time protection scans files as they are written, so the threat is stopped before anyone opens it. Notice that Defender and ClamAV give the *same file* different names — vendors label samples differently, which is why analysts correlate by hash rather than detection name.

Exercise 4 · Review the scan results

From the `clamscan` output, record: the file scanned, the detection name, the scan status, the number of infected files reported, and the signature database version shown in the scan summary.

Do not use `--remove` or `--move` options — you will scan this same file again in Lab 04.

STUDENT ASSIGNMENT

Submit a one-page analysis report containing:

- Three malware categories and their characteristics.
- The original and modified SHA-256 hashes.
- Results of the file identification exercise.
- EICAR scan results.
- An explanation of why the EICAR file is safe for training.
- Windows track: the Defender detection name and Protection history entry from Exercise 3b.

What you learned. Malware identification requires evidence, not assumptions based on filenames. You practiced file hashing, file type identification and antivirus testing.

Real-world application. Security operations center (SOC) analysts investigate suspicious email attachments and downloads daily. Hashes and antivirus results help them correlate alerts, compare samples, and decide whether deeper investigation is needed.

LAB 02

STATIC MALWARE ANALYSIS & FILE FORENSICS

Duration · 2–3 hours | Difficulty · Beginner | Linux · file, sha256sum, stat, strings, grep, xxd | Windows · Get-Item, strings64, Select-String, Format-Hex

Learning objectives

1. Perform static analysis without executing a suspicious file.
2. Extract readable strings.
3. Inspect file metadata.
4. Identify suspicious indicators.
5. Distinguish evidence from assumptions.
6. Document a static analysis investigation.

Background

Static analysis inspects a file without running it — the safest first look at anything suspicious. Common techniques include hashing, file format identification, string extraction, metadata inspection, header examination and signature comparison. In this lab you create a harmless file containing fictional indicators.

Exercise 1 · Create a training artifact

LINUX · TERMINAL

```
cd ~/MalwareLab/samples
cat > suspicious_sample.txt <<'EOF'
TRAINING_SAMPLE_ONLY
ApplicationName=InvoiceViewer
Version=1.0
Reference=INV-2026-1001
ObservedDomain=updates.training.invalid
ObservedIP=192.0.2.50
ObservedPath=/training/checkin
RegistryReference=HKCU\Software\TrainingDemo
PossibleAction=CreateTemporaryFile
PossibleAction=ContactRemoteServer
PossibleAction=CollectSystemInformation
END_TRAINING_SAMPLE
EOF
```

WINDOWS · POWERSHELL

```
cd $HOME\MalwareLab\samples
@'
TRAINING_SAMPLE_ONLY
ApplicationName=InvoiceViewer
Version=1.0
Reference=INV-2026-1001
ObservedDomain=updates.training.invalid
ObservedIP=192.0.2.50
ObservedPath=/training/checkin
RegistryReference=HKCU\Software\TrainingDemo
PossibleAction=CreateTemporaryFile
PossibleAction=ContactRemoteServer
PossibleAction=CollectSystemInformation
END_TRAINING_SAMPLE
'@ | Set-Content -Path suspicious_sample.txt -Encoding ascii
```

All indicators are fictional. The `.invalid` top-level domain is reserved and can never resolve, and `192.0.2.0/24` is a range reserved for documentation. Neither is real command-and-control infrastructure.

Exercise 2 · Identify the file

LINUX · TERMINAL

```
file suspicious_sample.txt
sha256sum suspicious_sample.txt
stat suspicious_sample.txt
```

WINDOWS · POWERSHELL

```
file suspicious_sample.txt
Get-FileHash -Path suspicious_sample.txt -Algorithm SHA256 | Format-List
Get-Item suspicious_sample.txt | Format-List Name, Length, CreationTime,
LastWriteTime, LastAccessTime, Attributes
(Get-Acl suspicious_sample.txt).Owner
Get-Item suspicious_sample.txt -Stream *
```

Windows result: `file` reports `ASCII text, with CRLF line terminators`; `Get-Item` shows the size and three timestamps; `Get-Acl` shows the owner (for example `MALWARELAB\student`). The last command lists the file's NTFS data streams — only `:$DATA` here. Files downloaded from the internet also carry a `Zone.Identifier` stream (the "Mark of the Web") recording where they came from — valuable evidence when tracing how an attachment arrived.

Record the filename, file type, size (in bytes, from `stat`), SHA-256 hash and modification time.

EXPECTED RESULT

`file` reports `ASCII text`; `stat` shows size, owner, permissions and Access/Modify/Change times.

WHY

These facts uniquely identify the sample and anchor it in time — the first lines of every analysis report, and how other analysts confirm they are looking at the same file.

Exercise 3 · Extract strings

1. List every readable string:

LINUX · TERMINAL

```
strings suspicious_sample.txt
```

WINDOWS · POWERSHELL

```
strings -nobanner suspicious_sample.txt
```

2. Filter for network-related indicators (`-E` allows the `|` "or" pattern; `-i` ignores case):

LINUX · TERMINAL

```
strings suspicious_sample.txt | grep -Ei \
  'domain|ip|http|server|path'
```

WINDOWS · POWERSHELL

```
strings -nobanner suspicious_sample.txt | Select-String -Pattern
'domain|ip|http|server|path'
```

Windows result: `Select-String` is case-insensitive by default and treats `|` as "or", so it matches the same four lines as `grep -Ei`.

3. Filter for action references:

LINUX · TERMINAL

```
strings suspicious_sample.txt | grep -i 'PossibleAction'
```

WINDOWS · POWERSHELL

```
Select-String -Path suspicious_sample.txt -Pattern 'PossibleAction'
```

Windows result: Three lines, each prefixed with the filename and line number (for example `suspicious_sample.txt:9:PossibleAction=CreateTemporaryFile`) — handy for citing evidence in a report.

EXPECTED RESULT

The network filter returns the `ObservedDomain`, `ObservedIP`, `ObservedPath` and `ContactRemoteServer` lines. The action filter returns three `PossibleAction=` lines.

WHY

Embedded domains, addresses and paths are common clues to what a program may do. Filtering turns a wall of text into a short list of leads — but a string shows only what a file *contains*, not what it *does*.

STUDENT QUESTIONS

1. Which strings suggest potential network communication?
2. Which strings suggest possible host activity?
3. Can a string alone prove that a program performs an action?
4. Why is the file considered a training artifact rather than confirmed malware?

Exercise 4 · Examine hexadecimal data

Display the first 128 bytes:

LINUX · TERMINAL

```
xxd -l 128 suspicious_sample.txt
```

If `xxd` is unavailable, use the built-in alternative:

LINUX · TERMINAL

```
od -Ax -tx1z -N128 suspicious_sample.txt
```

WINDOWS · POWERSHELL

```
Format-Hex -Path suspicious_sample.txt | Select-Object -First 8
```

Windows result: Offset, hexadecimal and ASCII columns; each row holds 16 bytes, so 8 rows = 128 bytes. Look for `0D 0A` at each line end — the CR LF pair Windows uses where the Linux dump shows only `0A`.

EXPECTED RESULT

Three columns: byte offset, hexadecimal values and the ASCII rendering. The first bytes, `5452 4149 4e49 4e47`, spell `TRAINING`.

WHY

Raw bytes cannot lie about format. Windows executables begin with `4d 5a` ("MZ") and Linux executables with `7f 45 4c 46` ("ELF"). Analysts check headers when names and strings may have been manipulated.

Exercise 5 · Build an indicator inventory

Recreate this table in your report, filling in your own hash.

INDICATOR	VALUE	CLASSIFICATION
Domain	`updates.training.invalid`	Fictional domain
IP address	`192.0.2.50`	Documentation address
Path	`/training/checkin`	Simulated network path
Registry reference	`HKCU\Software\TrainingDemo`	Static text reference
File hash	Student-generated	SHA-256 identifier

Important: these are embedded strings, not evidence that the file communicated with a network service or modified the registry.

STUDENT ASSIGNMENT

Prepare a static analysis report containing:

1. Sample identification
2. File metadata
3. SHA-256 hash
4. Extracted indicators
5. Observed strings
6. Possible behaviors suggested by those strings
7. Evidence limitations
8. Recommended next steps

What you learned. How to inspect a file without executing it, extract indicators, and distinguish an observed artifact from a hypothesis about malicious behavior.

Real-world application. Analysts use static analysis to triage attachments, identify embedded URLs, compare known samples and prioritize deeper investigation — and to write detection rules without running a potentially dangerous program.

LAB 03

SAFE BEHAVIOR SIMULATION & PROCESS MONITORING

Duration · 2–3 hours | Difficulty · Beginner | Linux · python3, ps, top, watch, strace | Windows · Get-Process, Task Manager, Procmon

Learning objectives

1. Observe running processes.
2. Monitor file creation.
3. Understand system calls.
4. Recognize how behavior differs from static evidence.
5. Record a process activity timeline.
6. Explain behavioral indicators used by endpoint detection systems.

Background

Dynamic analysis observes software while it runs. Malware may create files, launch processes, modify configurations or communicate over networks. Here you run a transparent Python simulation that only creates harmless local files. It contains no malware, establishes no persistence, contacts no external servers and collects no sensitive information. Confirm the VM is still isolated before you start.

Exercise 1 · Create the simulation program

1. Open a new file in the scripts folder with the nano editor:

LINUX · TERMINAL

```
cd ~/MalwareLab/scripts
nano behavior_simulator.py
```

WINDOWS · POWERSHELL

```
cd $HOME\MalwareLab\scripts
notepad behavior_simulator.py
```

Windows result: Notepad asks whether to create the file — choose **Yes**. After saving, run `Get-ChildItem` and confirm the name is `behavior_simulator.py`, not `behavior_simulator.py.txt`.

2. Type in the program below. Indentation matters in Python — use four spaces. In nano, save with **Ctrl+O**, Enter, then exit with **Ctrl+X**; in Notepad, save with **Ctrl+S**.

```
from pathlib import Path
from datetime import datetime, timezone
import os
import time

lab = Path.home() / "MalwareLab"
output = lab / "evidence" / "simulation"
output.mkdir(parents=True, exist_ok=True)

print("SAFE BEHAVIOR SIMULATION")
print(f"Process ID: {os.getpid()}")
print("Creating harmless local training files.")

for number in range(1, 6):
    filename = output / f"activity_{number}.txt"

    timestamp = datetime.now(
        timezone.utc
    ).isoformat()

    filename.write_text(
        f"Simulation event: {number}\n"
        f"Timestamp: {timestamp}\n"
        "No malicious activity performed.\n"
    )

    print(f"Created: {filename}", flush=True)
    time.sleep(3)

print("Simulation completed.")
```

Listing 1 — behavior_simulator.py. It writes five text files, three seconds apart, into `evidence/simulation/`.

Exercise 2 · Execute the simulation

LINUX · TERMINAL

```
python3 behavior_simulator.py
```

WINDOWS · POWERSHELL

```
python behavior_simulator.py
```

Windows result: The same output, with Windows paths such as `C:\Users\student\MalwareLab\evidence\simulation\activity_1.txt`.

EXPECTED RESULT

The program prints its process ID, five `Created:` lines about three seconds apart, then `Simulation completed.` — about 15 seconds in total.

WHY

You now know exactly what this program does, so you can check whether your monitoring tools see the same thing. Validating tools against known behavior is how analysts learn to trust them.

Exercise 3 · Monitor the process

1. Open a second terminal window. Start the simulator in the first, then immediately run this in the second:

LINUX · TERMINAL

```
ps aux | grep '[b]ehavior_simulator.py'
```

WINDOWS · POWERSHELL

Open a second PowerShell window. Start the simulator in the first, then immediately run:

```
Get-CimInstance Win32_Process -Filter "Name = 'python.exe'" | Select-Object
ProcessId, ParentProcessId, CommandLine
Get-Process -Name python | Format-Table Id, ProcessName, CPU, WorkingSet64, StartTime
Get-CimInstance Win32_Process -Filter "Name = 'python.exe'" | Invoke-CimMethod -
MethodName GetOwner
```

Task Manager is Windows' equivalent of `top`: press **Ctrl+Shift+Esc** → **Details**, right-click a column header → **Select columns** → tick **Command line**.

Windows result: One `python.exe` whose command line contains `behavior_simulator.py`, owned by `student`. `CPU` is total seconds of processor time (near zero) and `WorkingSet64` is memory in bytes. `ParentProcessId` points to your PowerShell window — parent–child relationships are a key detection signal (Word launching PowerShell, for example, is highly suspicious).

The brackets in `[b]ehavior` stop `grep` from matching its own command line. Record the process ID (PID), owner (USER), command, %CPU and %MEM.

2. Run the simulator again and view it live with `top` (press **q** to quit):

LINUX · TERMINAL

```
top
```

EXPECTED RESULT

One line showing your username, a PID matching the simulator's own output, near-zero CPU and a small memory percentage.

WHY

The PID ties every later observation to one specific process. Owner and resource use are early warning signs: an unexpected process running as root, or one pinning the CPU (as a cryptojacker would), deserves attention.

Exercise 4 · Monitor file activity

Start the simulator, and in the second terminal run:

LINUX · TERMINAL

```
watch -n 1 \
  'ls -lt ~/MalwareLab/evidence/simulation'
```

WINDOWS · POWERSHELL

```
while ($true) {
  Clear-Host
  Get-ChildItem $HOME\MalwareLab\evidence\simulation | Sort-Object LastWriteTime -
Descending | Format-Table Name, Length, LastWriteTime
  Start-Sleep -Seconds 1
}
```

Windows result: The listing refreshes every second as files appear. Press **Ctrl+C** to stop the loop.

Observe the creation and modification times as files appear. Press **Ctrl+C** to exit `watch`.

EXPECTED RESULT

The listing refreshes every second; `activity_1.txt` to `activity_5.txt` each get a fresh timestamp roughly three seconds apart, newest at the top.

WHY

Watching the filesystem shows the *effect* of a process. Rapid, repeated file changes are exactly the pattern defenders watch for when ransomware begins encrypting data.

Exercise 5 · Trace system calls

Programs ask the operating system kernel to do work through *system calls*. `strace` records them. `-f` follows child processes, `-e trace=file` keeps only file-related calls, and `-o` writes them to a log.

LINUX · TERMINAL

```
strace -f -e trace=file \
  -o ~/MalwareLab/logs/strace.log \
  python3 behavior_simulator.py

grep 'activity_' ~/MalwareLab/logs/strace.log
```

WINDOWS · POWERSHELL

Windows has no `strace`. Microsoft's **Process Monitor** (Procmon) uses a kernel driver to record every file, registry, network and process operation — the same "ground truth" idea.

1. Launch Procmon as administrator: `Start-Process C:\Tools\Sysinternals\Procmon64.exe -Verb RunAs`. Accept the licence.
2. Procmon starts capturing at once. Press **Ctrl+E** to stop, then **Ctrl+X** to clear the list.
3. Press **Ctrl+L** to open *Filter*. Add **Process Name · is · python.exe · Include**, click *Add*; then **Path · contains · activity_ · Include**, click *Add*, then *OK*.
4. On the toolbar, leave only **Show File System Activity** switched on (turn off registry, network, process/thread and profiling).
5. Press **Ctrl+E** to start capturing. In PowerShell run `python behavior_simulator.py`, wait for *Simulation completed.*, then press **Ctrl+E** again.
6. Choose **File → Save** → *Events displayed using current filter* → format **CSV** → path `C:\Users\student\MalwareLab\logs\procmon.csv`.

Review the capture from PowerShell:

```
Import-Csv $HOME\MalwareLab\logs\procmon.csv | Select-Object "Time of Day",
Operation, Path, Result | Format-Table -AutoSize
```

EXPECTED RESULT · WINDOWS

For each `activity_N.txt`: a `CreateFile` with Result `SUCCESS` (its Detail column shows *Desired Access: Generic Write* and *Disposition: OverwriteIf*), then `WriteFile` and `CloseFile`, possibly with some query operations — five groups about three seconds apart.

WHY

OverwriteIf means "create the file, or replace it if it exists" — Windows' version of Linux's `O_CREAT|O_TRUNC`. Procmon shows what the process actually asked the kernel to do. The filters matter: unfiltered, Procmon records tens of thousands of events a minute, and narrowing by process, path and operation is a core analyst skill.

EXPECTED RESULT

Lines similar to `openat(AT_FDCWD, "/home/student/MalwareLab/evidence/simulation/activity_1.txt", O_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC, 0666) = 3` — one per file. Exact calls vary with OS and Python version.

WHY

`O_CREAT` means "create the file" and `O_WRONLY` means "open for writing". System calls are ground truth: they show what a process actually asked the OS to do, regardless of what it printed.

STUDENT QUESTIONS

1. What is a process ID?
2. What information does `strace` provide?
3. Which file operations did the program perform?
4. Why would file activity be useful during malware analysis?
5. Does creating multiple files automatically indicate malicious behavior?

Exercise 6 · Create a behavior timeline

Record events in this format, using the actual timestamps from your run (the file contents and `ls -l --time-style=full-iso` both give exact times).

WINDOWS · POWERSHELL

```
Get-ChildItem $HOME\MalwareLab\evidence\simulation | Select-Object Name,
CreationTimeUtc, LastWriteTimeUtc
```

TIME	EVENT	EVIDENCE SOURCE	OBSERVATION
—	Program starts	Terminal / process list	Python process begins
—	File creation	strace / filesystem	First training file created
—	Additional file activity	Filesystem	Remaining files created
—	Program exits	Terminal	Simulation completes

STUDENT ASSIGNMENT

Submit:

- Process information.
- Screenshot of the running simulation.
- The five generated filenames.
- Relevant `strace` output.
- Windows track: the Procmon CSV extract in place of `strace` output.
- A timeline of observed activity.
- A paragraph distinguishing harmless simulation behavior from potentially malicious activity.

What you learned. How to observe program behavior and correlate process activity with filesystem events — and that a behavior may be suspicious in context without being inherently malicious.

Real-world application. Endpoint detection and response (EDR) systems monitor process execution and filesystem changes. Defenders may investigate an unusual burst of file changes associated with ransomware, but must correlate it with additional evidence before concluding an attack occurred.

LAB 04

MALWARE DETECTION USING YARA & CLAMAV

Duration · 2–3 hours | Difficulty · Beginner | Linux · yara, clamscan | Windows · yara64, clamscan, Start-MpScan

Learning objectives

1. Explain signature-based detection.
2. Create a basic YARA rule.
3. Scan files using YARA.
4. Test antivirus detection.
5. Understand false positives and false negatives.
6. Improve detection rules using multiple indicators.

Background

YARA is a pattern-matching tool used by malware researchers. You write *rules* describing patterns that identify suspicious files or known malware families. **ClamAV** is an open-source antivirus engine that scans files against a large, maintained signature database. YARA is for writing your own detections; ClamAV applies detections others have written.

A YARA rule has three sections: `meta` (descriptive information), `strings` (the patterns to look for) and `condition` (the logic that decides a match).

Exercise 1 · Create your first YARA rule

LINUX · TERMINAL

```
cd ~/MalwareLab/rules
nano training_sample.yar
```

WINDOWS · POWERSHELL

```
cd $HOME\MalwareLab\rules
notepad training_sample.yar
```

Windows result: Choose **Yes** to create the file. If it is saved as `training_sample.yar.txt`, fix it with `Rename-Item training_sample.yar.txt training_sample.yar`.

Enter the rule, then save (nano: Ctrl+O, Enter, Ctrl+X · Notepad: Ctrl+S):

```
rule Training_Sample_Detection
{
    meta:
        description = "Detects the local training artifact"
        author = "Cybersecurity Masterclass"
        category = "Training"
        severity = "Informational"

    strings:
        $marker = "TRAINING_SAMPLE_ONLY"
        $domain = "updates.training.invalid"
        $action = "PossibleAction=CreateTemporaryFile"

    condition:
        $marker and $domain and $action
}
```

Listing 2 — training_sample.yar. The condition requires all three strings to be present.

Exercise 2 · Scan the training artifact

LINUX · TERMINAL

```
yara \
  ~/MalwareLab/rules/training_sample.yar \
  ~/MalwareLab/samples/suspicious_sample.txt
```

WINDOWS · POWERSHELL

```
yara $HOME\MalwareLab\rules\training_sample.yar
$HOME\MalwareLab\samples\suspicious_sample.txt
```

Windows result: `Training_Sample_Detection C:\Users\student\MalwareLab\samples\suspicious_sample.txt`

EXPECTED RESULT

YARA prints the rule name followed by the file path: `Training_Sample_Detection /home/student/MalwareLab/samples/suspicious_sample.txt`. Add `-s` to see which strings matched.

WHY

The file contains all three patterns, so the condition is true. A rule should always be tested against a known positive before it is trusted.

Exercise 3 · Test a nonmatching file

LINUX · TERMINAL

```
yara \
  ~/MalwareLab/rules/training_sample.yar \
  ~/MalwareLab/samples/training.txt
```

WINDOWS · POWERSHELL

```
yara $HOME\MalwareLab\rules\training_sample.yar $HOME\MalwareLab\samples\training.txt
```

EXPECTED RESULT

No output. YARA prints nothing when no rule matches.

WHY

Testing a known negative proves the rule is not matching everything. Silence means "no match" — not "error" — so learn to read it correctly.

Exercise 4 · Understand false positives

1. Create a file containing only the generic marker:

LINUX · TERMINAL

```
cat > ~/MalwareLab/samples/partial_match.txt <<'EOF'
TRAINING_SAMPLE_ONLY
This is an ordinary training document.
It does not contain the other indicators.
EOF
```

WINDOWS · POWERSHELL

```
@'
TRAINING_SAMPLE_ONLY
This is an ordinary training document.
It does not contain the other indicators.
'@ | Set-Content -Path $HOME\MalwareLab\samples\partial_match.txt -Encoding ascii
```

2. Scan it — no match should occur, because the rule requires all three strings:

LINUX · TERMINAL

```
yara \
  ~/MalwareLab/rules/training_sample.yar \
  ~/MalwareLab/samples/partial_match.txt
```

WINDOWS · POWERSHELL

```
yara $HOME\MalwareLab\rules\training_sample.yar
$HOME\MalwareLab\samples\partial_match.txt
```

3. Open the rule in your editor (nano or Notepad) and temporarily change the condition to:

```
condition:
    $marker
```

4. Save and scan `partial_match.txt` again.

EXPECTED RESULT

With the original condition: no match. With the loosened condition: the ordinary document now matches.

WHY

This is a **false positive** — a harmless file flagged as suspicious. Rules built on one generic string fire on innocent files and bury analysts in noise. Requiring several distinctive indicators together makes a rule precise.

5. Restore the original condition — `$marker and $domain and $action` — and save. Lab 05 depends on it.

Exercise 5 · Scan using ClamAV

LINUX · TERMINAL

```
clamscan ~/MalwareLab/samples/eicar.com
clamscan ~/MalwareLab/samples/training.txt
```

WINDOWS · POWERSHELL

```
clamscan $HOME\MalwareLab\samples\eicar.com
clamscan $HOME\MalwareLab\samples\training.txt
```

EXPECTED RESULT

`eicar.com` is reported `FOUND` (Infected files: 1). `training.txt` is reported `OK` (Infected files: 0).

WHY

ClamAV detects EICAR because its database contains that signature — and does not flag your YARA training artifact, because nobody published a signature for it. Signature engines only know what they have been taught. If EICAR is not detected, re-check the database (Part 2, Step 6).

Exercise 5b · Scan with Microsoft Defender Windows only

Run a Defender custom scan of the samples folder — which contains `eicar.com` — from an **administrator** PowerShell window:

WINDOWS · POWERSHELL

```
Start-MpScan -ScanType CustomScan -ScanPath "$HOME\MalwareLab\samples"
Get-MpThreatDetection | Sort-Object InitialDetectionTime -Descending | Select-Object
-First 3 | Format-List InitialDetectionTime, Resources
Get-MpPreference | Select-Object -ExpandProperty ExclusionPath
```

EXPECTED RESULT · WINDOWS

The scan completes with **no new detection**, even though `eicar.com` is in the folder and ClamAV flagged it moments ago. The only detections listed are from Exercise 3b in Lab 01. The last command shows why: the folder is excluded.

WHY

This is a **false negative caused by configuration**, not by clever malware — Defender never looked. It is why attackers with administrator rights add exclusions, and why responders review the exclusion list early in an investigation. Leave the exclusion in place; Lab 05 still uses the lab folder.

Exercise 6 · Understand detection limitations

Discuss in class or in writing:

- Why a YARA match is not automatically proof of malicious behavior.
- Why known malware may be detected by signatures.
- Why new or modified malware may evade existing signatures — a **false negative**.
- Why analysts combine signatures with behavioral and contextual evidence.

STUDENT ASSIGNMENT

Submit:

1. The completed YARA rule.
2. A screenshot of a successful match.
3. A screenshot of a nonmatching scan.
4. ClamAV scan results.
5. An explanation of false positives.
6. An explanation of false negatives.
7. A recommendation for improving detection quality.
8. Windows track: the Exercise 5b scan result and an explanation of why the exclusion produced a false negative.

What you learned. How to create and test simple detection rules, and why results must be interpreted carefully — broad rules can identify harmless files.

Real-world application. Security teams use YARA for malware classification, threat hunting and forensic investigations. Antivirus and endpoint products use signatures alongside other techniques to detect known threats and suspicious behavior.

LAB 05

INCIDENT RESPONSE & FORENSIC REPORTING

Duration · 3 hours | Difficulty · Beginner | Linux · python3, jq, sha256sum, yara | Windows · ConvertFrom-Json, Get-FileHash, Get-WinEvent

Learning objectives

1. Investigate a simulated malware incident.
2. Collect and preserve evidence.
3. Identify indicators of compromise.
4. Build an incident timeline.
5. Recommend containment and remediation.
6. Produce a professional incident response report.

SCENARIO

You are a junior analyst in a Security Operations Center. A workstation generated an alert after a suspicious file was discovered. The initial report includes a suspicious attachment name, a fictional external domain, a simulated file creation event, a training detection alert and a possible unauthorized process execution.

Your assignment: determine what the evidence *actually* supports.

This is a simulated incident. No actual infection or compromise has occurred.

Exercise 1 · Generate the incident evidence

LINUX · TERMINAL

```
cd ~/MalwareLab/scripts
nano generate_incident.py
```

WINDOWS · POWERSHELL

```
cd $HOME\MalwareLab\scripts
notepad generate_incident.py
```

```
from pathlib import Path
import json

base = Path.home() / "MalwareLab"
incident = base / "evidence" / "incident-001"
incident.mkdir(parents=True, exist_ok=True)

events = [
    {
        "timestamp": "2026-11-09T10:00:00Z",
        "event": "FileObserved",
        "host": "TRAINING-WS01",
        "file": "invoice_attachment.txt"
    },
    {
        "timestamp": "2026-11-09T10:01:00Z",
        "event": "ProcessObserved",
        "host": "TRAINING-WS01",
        "process": "training_simulator"
    },
    {
        "timestamp": "2026-11-09T10:02:00Z",
        "event": "FileCreated",
        "host": "TRAINING-WS01",
        "file": "activity_1.txt"
    },
    {
        "timestamp": "2026-11-09T10:03:00Z",
        "event": "NetworkIndicatorReferenced",
        "host": "TRAINING-WS01",
        "domain": "updates.training.invalid"
    },
    {
        "timestamp": "2026-11-09T10:04:00Z",
        "event": "TrainingAlert",
        "host": "TRAINING-WS01",
        "signature": "Training_Sample_Detection"
    }
]

with (incident / "events.json").open("w") as f:
    json.dump(events, f, indent=2)

(incident / "invoice_attachment.txt").write_text(
    "TRAINING_SAMPLE_ONLY\n"
    "updates.training.invalid\n"
```

```
    "PossibleAction=CreateTemporaryFile\n"
)

print("Incident evidence generated:")
print(incident)
```

Listing 3 — generate_incident.py. It writes fictional records only; it does not launch the named process or make a network connection.

LINUX · TERMINAL

```
python3 generate_incident.py
```

WINDOWS · POWERSHELL

```
python generate_incident.py
```

Windows result: The script prints `C:\Users\student\MalwareLab\evidence\incident-001`.

Expected result: the script prints the path `/home/<you>/MalwareLab/evidence/incident-001`, which now contains `events.json` and `invoice_attachment.txt`.

Exercise 2 · Examine the evidence

1. View the raw event file:

LINUX · TERMINAL

```
cat ~/MalwareLab/evidence/incident-001/events.json
```

WINDOWS · POWERSHELL

```
Get-Content $HOME\MalwareLab\evidence\incident-001\events.json
```

2. Use `jq` to turn each event into one tab-separated line:

LINUX · TERMINAL

```
jq -r \
  '.[] | [.timestamp, .event, .host] | @tsv' \
  ~/MalwareLab/evidence/incident-001/events.json
```

WINDOWS · POWERSHELL

```
$events = Get-Content $HOME\MalwareLab\evidence\incident-001\events.json -Raw |
ConvertFrom-Json
$events | Select-Object timestamp, event, host | Format-Table -AutoSize
```

Windows result: A five-row table, oldest first. `ConvertFrom-Json` turns each record into an object whose fields you can sort and filter. PowerShell 7 may display the timestamps converted to your local time zone — always report times in UTC, as recorded in the file. Because jq was installed in Step 4a, the Linux `jq` command also works unchanged with a Windows path.

EXPECTED RESULT

Five lines, oldest first, for example `2026-11-09T10:00:00Z FileObserved TRAINING-WS01.`

WHY

Security tools export events as JSON. `jq` reduces them to the fields that matter, making patterns and sequence easy to read.

From the output, identify: the hostname, the earliest event, the file referenced, the process referenced, the network indicator and the detection name.

Exercise 3 · Preserve evidence

LINUX · TERMINAL

```
cd ~/MalwareLab/evidence/incident-001

sha256sum events.json invoice_attachment.txt \
  > evidence_hashes.sha256

sha256sum -c evidence_hashes.sha256
```

WINDOWS · POWERSHELL

```
cd $HOME\MalwareLab\evidence\incident-001

Get-FileHash -Path events.json, invoice_attachment.txt -Algorithm SHA256 |
  Select-Object Hash, @{ Name = 'File'; Expression = { Split-Path $_.Path -Leaf } } |
  Export-Csv evidence_hashes.csv -NoTypeInformation

Import-Csv evidence_hashes.csv | ForEach-Object {
  $current = (Get-FileHash -Path $_.File -Algorithm SHA256).Hash
  $status = if ($current -eq $_.Hash) { 'OK' } else { 'FAILED' }
  "$($_.File): $status"
}
```

Windows result: `events.json: OK` and `invoice_attachment.txt: OK`. The first command records the hashes in `evidence_hashes.csv`; the second re-hashes each file and compares — the same job `sha256sum -c` does.

EXPECTED RESULT

`events.json: OK` and `invoice_attachment.txt: OK`.

WHY

Recording hashes at collection time lets you prove later that evidence has not changed — if one byte were altered, the check would report `FAILED`. In a real investigation also record collection time, source system, collector identity and handling history (chain of custody).

Exercise 4 · Perform a YARA scan

LINUX · TERMINAL

```
yara \
  ~/MalwareLab/rules/training_sample.yar \
  invoice_attachment.txt
```

WINDOWS · POWERSHELL

```
yara $HOME\MalwareLab\rules\training_sample.yar invoice_attachment.txt
```

EXPECTED RESULT

`Training_Sample_Detection invoice_attachment.txt`

WHY

The attachment contains the three strings the rule requires, so the alert is reproducible — you confirmed it independently. But a match confirms content, not a real infection. Record this distinction in your report.

Exercise 4b · Collect real Windows telemetry Windows only

The records in `events.json` are fictional, but your VM holds *genuine* logs from the earlier labs. Real investigations correlate exactly these sources. In an **administrator** PowerShell window:

WINDOWS · POWERSHELL

```
Get-WinEvent -LogName "Microsoft-Windows-Windows Defender/Operational" -MaxEvents 300
|
  Where-Object { $_.Id -in 1116, 1117, 5007 } |
  Select-Object TimeCreated, Id, @{ Name = 'Summary'; Expression = { ($_.Message -
split '\r?\n')[0] } } |
  Format-Table -Wrap
```

EVENT ID	MEANING	WHERE IT CAME FROM
1116	Malware detected	Lab 01, Exercise 3b
1117	Action taken (quarantine or remove)	Lab 01, Exercise 3b
5007	Defender configuration changed	The exclusion added in Part 2, Step 7

EXPECTED RESULT · WINDOWS

1116 and 1117 events stamped with the time of Exercise 3b, and a 5007 event recording the exclusion. If no 5007 appears, increase `-MaxEvents`.

WHY

Unlike the training records, these were written by the operating system itself — independent evidence. A 5007 event adding an exclusion shortly before a detection gap is a classic sign of defense tampering. Add these events to your evidence inventory, clearly labelled as genuine lab telemetry, and use them when answering "What additional logs would you request?"

Exercise 5 · Reconstruct the timeline

TIME (UTC)	EVENT	INTERPRETATION
10:00	FileObserved	Suspicious attachment recorded
10:01	ProcessObserved	Process event recorded
10:02	FileCreated	File creation recorded
10:03	NetworkIndicatorReferenced	Domain appears in evidence
10:04	TrainingAlert	Simulated detection recorded

The domain reference does not prove network communication. The process event is a fictional record, not independently verified execution.

STUDENT QUESTIONS

1. Which event occurred first?
2. What evidence supports the alert?
3. Was actual network communication established?
4. What additional logs would you request?
5. What information is missing before declaring a confirmed compromise?

Exercise 6 · Develop an incident response plan

Write a proposed plan using the six phases below. You do not isolate or alter any real systems.

PHASE	WHAT HAPPENS
1 · Preparation	Confirm the team has appropriate tools, procedures and authorization.
2 · Identification	Review alerts, host information, files and logs. Decide whether this is a real incident or a false positive.
3 · Containment	If compromise is confirmed or strongly suspected, isolate affected endpoints using approved procedures.
4 · Eradication	Remove confirmed malicious components and fix the original security weakness.
5 · Recovery	Restore trusted systems, validate controls and monitor for recurrence.
6 · Lessons learned	Document what happened, what worked and which improvements are needed.

Exercise 7 · Write the incident report

Use this template as the model for your report.

MALWARE INCIDENT INVESTIGATION REPORT

Incident ID	MAL-TRAINING-001
Affected host	TRAINING-WS01
Classification	Simulated suspicious file activity
Incident status	Training exercise
Severity	Informational (simulation)

EXECUTIVE SUMMARY

A fictional workstation generated a training alert associated with a suspicious file. The evidence includes file references, a simulated process event, a fictional domain indicator and a training detection signature. It is sufficient for a practice investigation but does not establish that real malware executed.

EVIDENCE COLLECTED

- Simulated event log
- Training attachment
- SHA-256 hashes
- YARA scan result
- Reconstructed event timeline

INDICATORS

- Filename: `invoice_attachment.txt`
- Domain: `updates.training.invalid`
- Process reference: `training_simulator`
- Detection: `Training_Sample_Detection`

FINDINGS

The attachment contains the strings required by the training YARA rule. The event records describe a possible execution sequence, but they are simulated and do not independently demonstrate malicious execution or network activity.

RECOMMENDED ACTIONS

- Validate suspicious files using approved security tools.
- Correlate endpoint and network telemetry.
- Preserve evidence.
- Investigate the source of suspicious attachments.
- Apply appropriate containment if actual malicious activity is confirmed.
- Review endpoint detection and email security controls.

FINAL ASSESSMENT

This was a controlled training scenario. No actual malware infection was demonstrated.

STUDENT ASSIGNMENT

Submit a complete incident report containing:

1. Executive summary
2. Incident timeline
3. Evidence inventory
4. Hash verification results
5. YARA results
6. Indicators and their limitations
7. Assessment of whether compromise was established
8. Recommended containment and remediation
9. Lessons learned
10. Windows track: the Exercise 4b event-log extract, with interpretation

What you learned. How to organize an investigation, preserve evidence, correlate events and communicate findings — and that a responsible analyst distinguishes confirmed facts from simulated events, hypotheses and incomplete evidence.

Real-world application. SOCs and incident response teams use these techniques to investigate suspicious attachments, endpoint alerts and ransomware. Accurate reporting lets organizations make defensible decisions about containment, recovery and security improvements.

ASSESSMENT

FINAL COURSE ASSESSMENT

Capstone project

Complete a simulated malware investigation using the laboratory tools, from a fresh restore of the `Malware-Lab-Clean` snapshot.

DELIVERABLE	REQUIREMENT
1 · File identification	Identify the file type, calculate its hash and document relevant metadata.
2 · Static analysis	Extract indicators and explain their significance.
3 · Behavioral analysis	Run the safe behavior simulator and record process and file activity.
4 · Detection	Create and execute a YARA rule and perform a ClamAV scan.
5 · Incident response	Prepare an incident report with evidence, findings, limitations and recommendations.

Grading rubric

CATEGORY	POINTS
Safe laboratory setup	10
File identification and hashing	15
Static analysis	20
Behavioral analysis	20
YARA and antivirus detection	15
Incident report and recommendations	20
Total	**100**

Suggested passing score: 80%.

Windows-track students submit PowerShell output, the Procmon extract and the Defender evidence in place of the Linux equivalents. Grading is identical.

FOR INSTRUCTORS

INSTRUCTOR GUIDANCE

Suggested teaching method

For every laboratory:

1. Introduce the cybersecurity concept.
2. Explain why it matters in the real world.
3. Demonstrate the first exercise.
4. Allow students to complete the remaining exercises.
5. Have students record evidence.
6. Discuss the observations.
7. Explain how defenders use the technique.
8. Review the assignment and knowledge-check questions.

Common troubleshooting

PROBLEM	RESOLUTION
ClamAV cannot scan files	Verify ClamAV is installed and its signature database is available (`clamscan --version` shows a database number).
FreshClam says another update is running	The distribution's updater service is already active. Stop it before a manual update, or let it finish.
YARA reports a syntax error	Check the rule's braces, quotation marks, string declarations and condition. The error message gives the line number.
Python script fails	Verify the Python version, file location and indentation (spaces, not tabs).
strace reports permission problems	Trace only the student's own simulation process, launched by strace itself, and check local security restrictions.
The simulation finishes too quickly	Start monitoring immediately, or increase `time.sleep(3)` in the script.
The virtual machine has no internet	Expected after isolation. Install tools and update signatures before isolating.
Something went wrong and the VM is messy	Restore the `Malware-Lab-Clean` snapshot and repeat Part 2, Step 8.
Windows: `python` opens the Microsoft Store	Settings → Apps → Advanced app settings → App execution aliases → turn off the *python.exe* and *python3.exe* aliases, then reopen PowerShell.
Windows: "running scripts is disabled on this system"	Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (Part 2, Step 4e).
Windows: `yara`, `file` or `strings` "not recognized"	The profile is missing or not loaded. Check `Test-Path $PROFILE`, compare it with Step 4e and reopen PowerShell.
Windows: eicar.com disappears or Python raises PermissionError	Defender removed it. Add the exclusion from Part 2, Step 7.
Windows: FreshClam "Can't open/parse the config file"	`freshclam.conf` is missing from `C:\Program Files\ClamAV` or still contains the *Example* line.
Windows: Notepad saved a file as `.txt`	Turn on file name extensions in File Explorer and fix it with `Rename-Item`.
Windows: Procmon shows thousands of events	Apply the filters (Ctrl+L): Process Name is python.exe, Path contains activity_.

PROBLEM	RESOLUTION
Windows: setup says "This PC can't run Windows 11"	The VM lacks TPM 2.0 or Secure Boot. Recreate it with Version *Windows 11 (64-bit)* in VirtualBox 7 or later.

FURTHER STUDY

RECOMMENDED STUDY RESOURCES

RESOURCE	WHAT IT OFFERS
MITRE ATT&CK	A knowledge base of adversary tactics and techniques.
MITRE ATT&CK Software	Documented malware and adversary software.
CISA	Government cybersecurity guidance, malware analysis reports and defensive resources.
YARA documentation	Rule syntax for the classic YARA used in these labs. The newer YARA-X may differ slightly.
ClamAV documentation	Installation, configuration and scanning.
Wireshark documentation	Network protocol analysis and packet inspection.
NIST SP 800-61	Incident response guidance.
Sysinternals	Documentation for Process Monitor, Strings and the rest of the suite.
Microsoft Defender Antivirus	How Windows' built-in antivirus detects, logs and is configured.
PowerShell documentation	Reference for every cmdlet used in the Windows track.

COURSE COMPLETION SUMMARY

By completing these five labs you have practiced the foundational workflow of a malware analyst. You now understand:

- How malware differs from legitimate software.
- How to identify file types and calculate hashes.
- How to inspect suspicious files without executing them.
- How to observe safe program behavior.
- How to write basic YARA detection rules.
- How antivirus scanning works.
- How to preserve and interpret evidence.
- How to create a professional incident response report.

Most importantly: effective malware analysis requires careful observation, accurate evidence handling, sound technical judgment and safe laboratory practices.

CYBERSECURITY MASTERCLASS BY JOSHUA MOSES · CYBERTRACE FOUNDATION